

HO12: Processamento e Otimização de Consulta

Felipe Gabriel de Carvalho

(1) Apresentar a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem natural (da esquerda para a direita), (2) a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem reversa (da direita para a esquerda), (3) a árvore de consulta otimizada, (4) a consulta reescrita de acordo com a árvore de consulta otimizada com o *parsing* da consulta em ordem natural (da esquerda para a direita), (5) a consulta reescrita de acordo com a árvore de consulta otimizada com o *parsing* da consulta em ordem reversa (da direita para a esquerda) e (6) o plano de execução da consulta otimizada para cada uma das consultas SQL apresentadas abaixo:

SELECT A.CPF, A.Nome, B.Nome

FROM Funcionarios A, Clientes B, Aluguel C, Funcionarios D

WHERE A.CPF=B.CPF

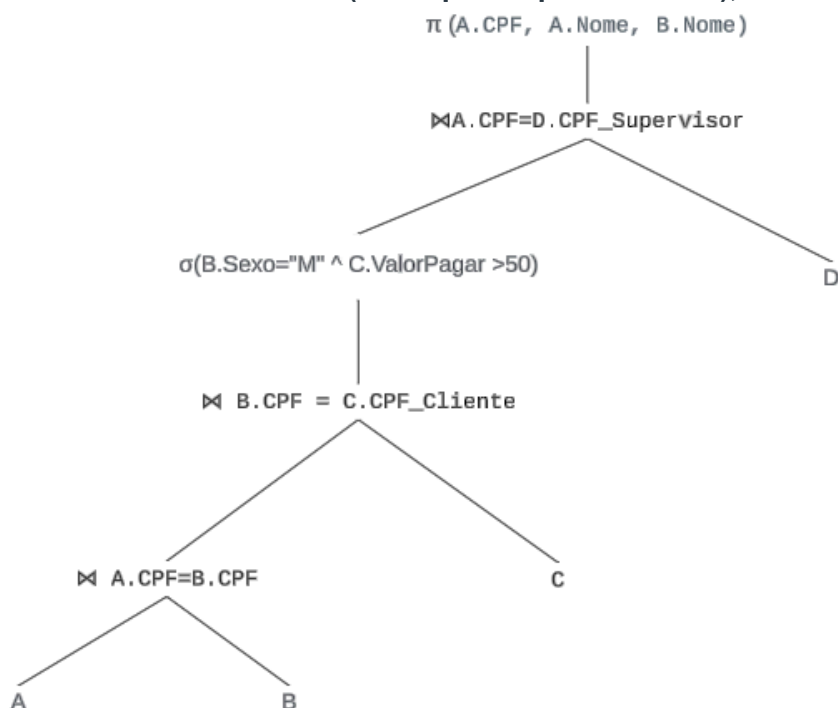
AND B.CPF=C.CPF_Cliente

AND B.Sexo="M"

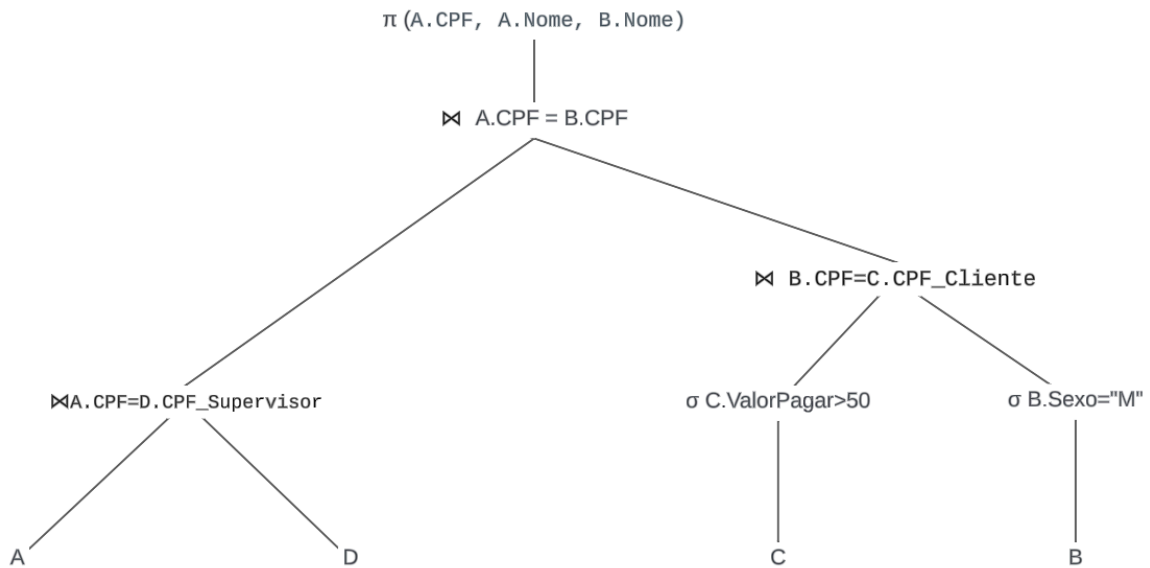
AND C.ValorPagar>50

AND A.CPF=D.CPF_Supervisor

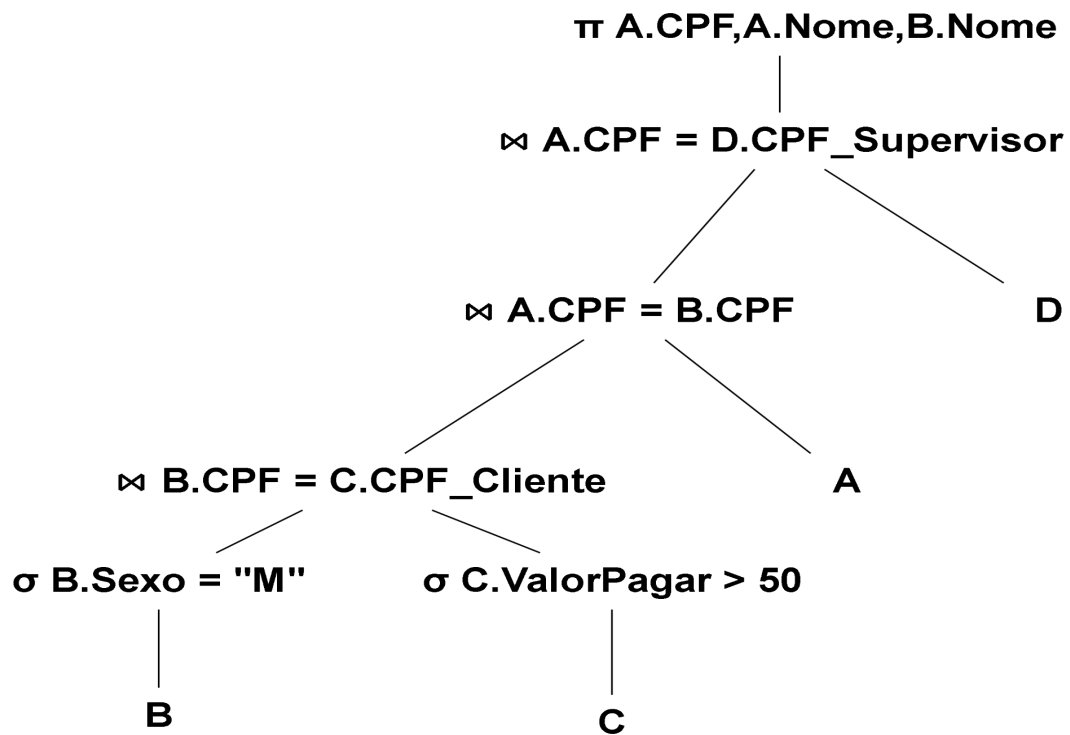
(1) Apresentar a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem natural (da esquerda para a direita),



(2) a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem reversa (da direita para a esquerda)



(3) a árvore de consulta otimizada,



(4) a consulta reescrita de acordo com a árvore de consulta otimizada com o *parsing* da consulta em ordem natural (da esquerda para a direita):

```
SELECT A.CPF, A.Nome, B.Nome
FROM Funcionarios A
JOIN Clientes B ON A.CPF = B.CPF
JOIN Aluguel C ON B.CPF = C.CPF_Cliente
JOIN Funcionarios D ON A.CPF = D.CPF_Supervisor
WHERE B.Sexo = 'M'
AND C.ValorPagar > 50;
```

(5) a consulta reescrita de acordo com a árvore de consulta otimizada com o *parsing* da consulta em ordem reversa (da direita para a esquerda):

```
SELECT A.CPF, A.Nome, B.Nome
FROM Funcionarios A
JOIN Funcionarios D ON A.CPF = D.CPF_Supervisor
JOIN Clientes B ON A.CPF = B.CPF
JOIN Aluguel C ON B.CPF = C.CPF_Cliente
WHERE B.Sexo = 'M'
AND C.ValorPagar > 50;
```

(6) o plano de execução da consulta otimizada para cada uma das consultas SQL

1. Pesquisa Linear → arquivo não indexado B (Clientes)
2. Pesquisa Linear → arquivo não indexado C (Aluguel)
3. Junção de loop único → varre arquivo não indexado B (Clientes) e pesquisa arquivo de índice de C (Aluguel)
4. Junção de loop único → varre arquivo não indexado A (Funcionarios) e pesquisa resultado intermediário ($B \bowtie C$)
5. Junção de loop único → varre resultado intermediário ($A \bowtie B \bowtie C$) e pesquisa arquivo de índice D (Funcionarios)
6. Projeção final (π)
 - Colunas: A.CPF, A.Nome, B.Nome

```
SELECT A.Nome, C.Nome
FROM Filmes A, AtoresEmFilmes B, Atores C, Midias D
WHERE A.Codigo=B.CodFilme
```

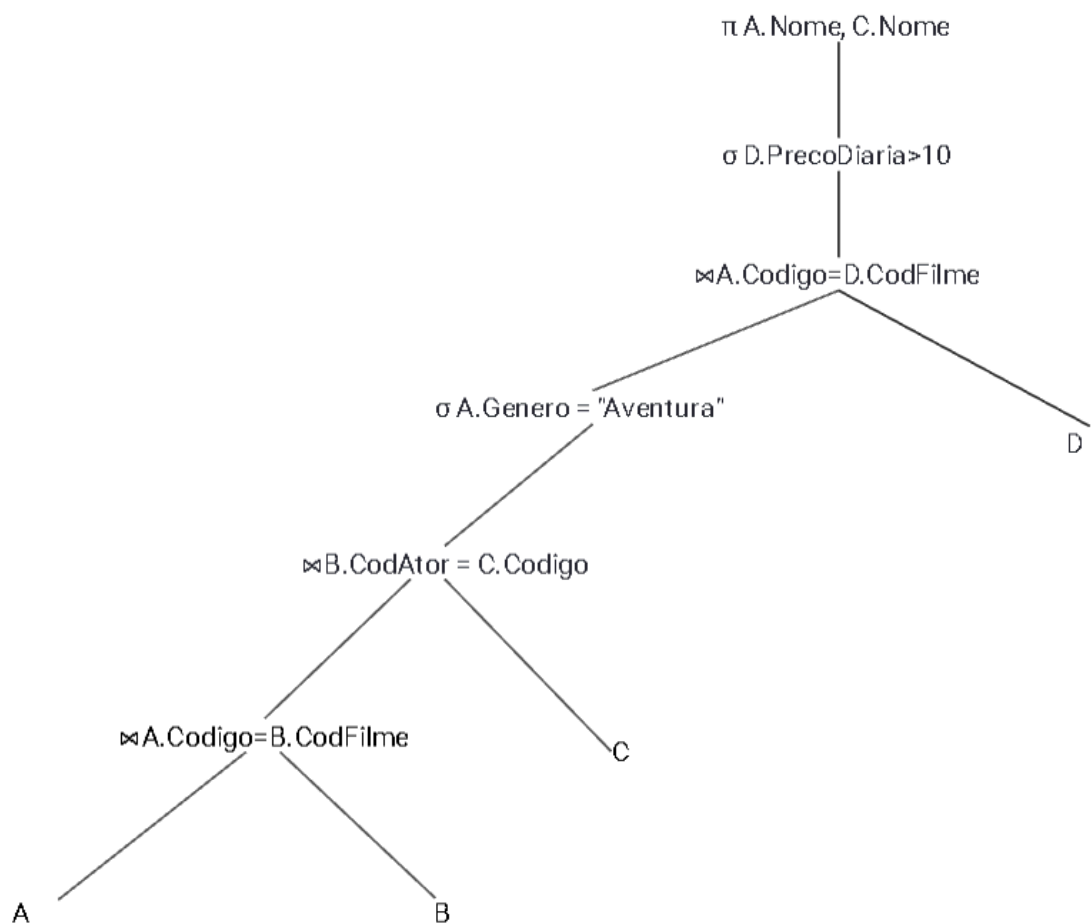
AND B.CodAtor=C.Codigo

AND A.Genero="Aventura"

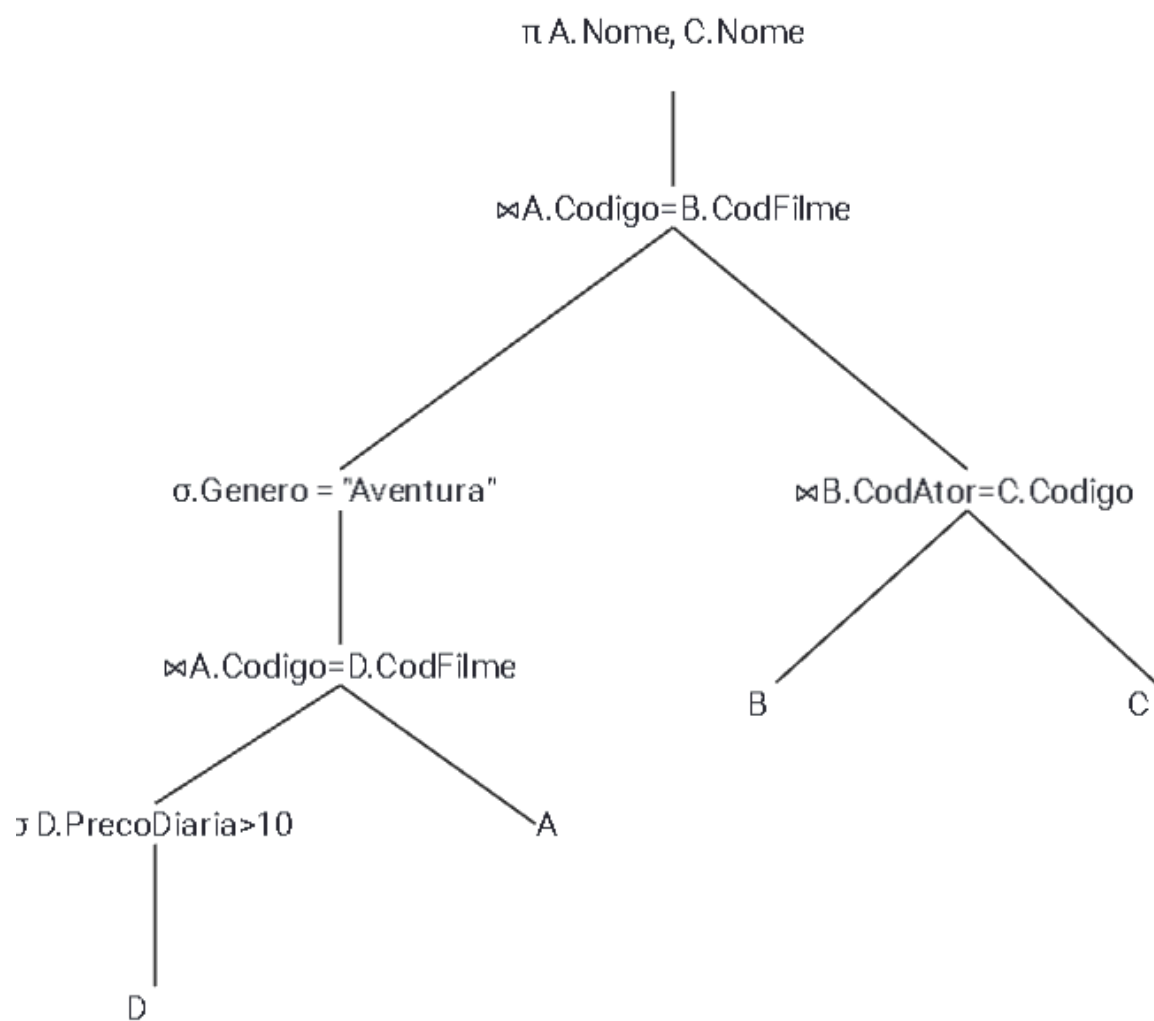
AND A.Codigo=D.CodFilme

AND D.PrecoDiaria>10

(1) Apresentar a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem natural (da esquerda para a direita):



(2) a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem reversa (da direita para a esquerda):



(3) a árvore de consulta otimizada:

```
SELECT J.NomeFilme, K.NomeAtor
FROM (
```

```

SELECT H.CodFilme, C.Nome AS NomeAtor
FROM AtoresEmFilmes AS H
JOIN Atores AS C ON H.CodAtor = C.Codigo
) AS K
JOIN (
    SELECT F.CodFilme, A.Nome AS NomeFilme
    FROM Midias AS F
    JOIN Filmes AS A ON F.CodFilme = A.Codigo
    WHERE F.PrecoDiaria > 10 AND A.Genero = 'Aventura'
) AS J
ON J.CodFilme = K.CodFilme;

```

(6) o plano de execução da consulta otimizada para cada uma das consultas SQL apresentadas abaixo:

1. Pesquisa Linear → arquivo não indexado Midias (F)
2. Pesquisa Linear → arquivo não indexado Filmes (A)
3. Junção de loop único → varre arquivo não indexado Midias (F) e pesquisa arquivo de índice de Filmes (A)
4. Pesquisa Linear → arquivo não indexado AtoresEmFilmes (H)
5. Pesquisa Linear → arquivo não indexado Atores (C)
6. Junção de loop único → varre arquivo não indexado AtoresEmFilmes (H) e pesquisa arquivo de índice de Atores (C)
7. Junção de loop único → varre resultado intermediário J e pesquisa resultado intermediário K
8. Projeção final (π) - Colunas: J.NomeFilme, K.NomeAtor

```

SELECT A.CPF, A.Nome, B.Nome
FROM Funcionarios A, Clientes B, Aluguel C, Pagamentos D
WHERE A.CPF=B.CPF
AND C.ValorPagar>100

```

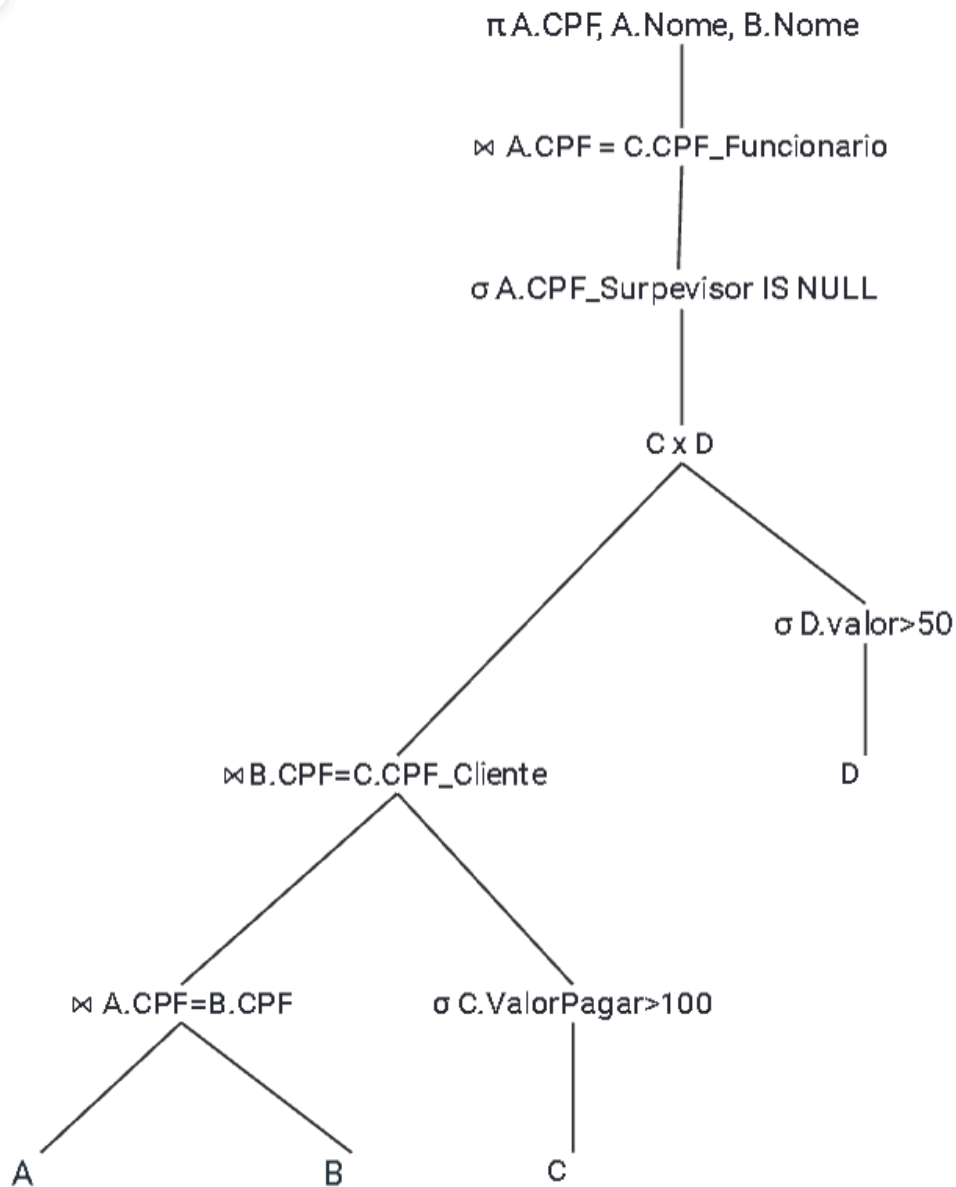
AND B.CPF=C.CPF_Cliente

AND D.Valor<50

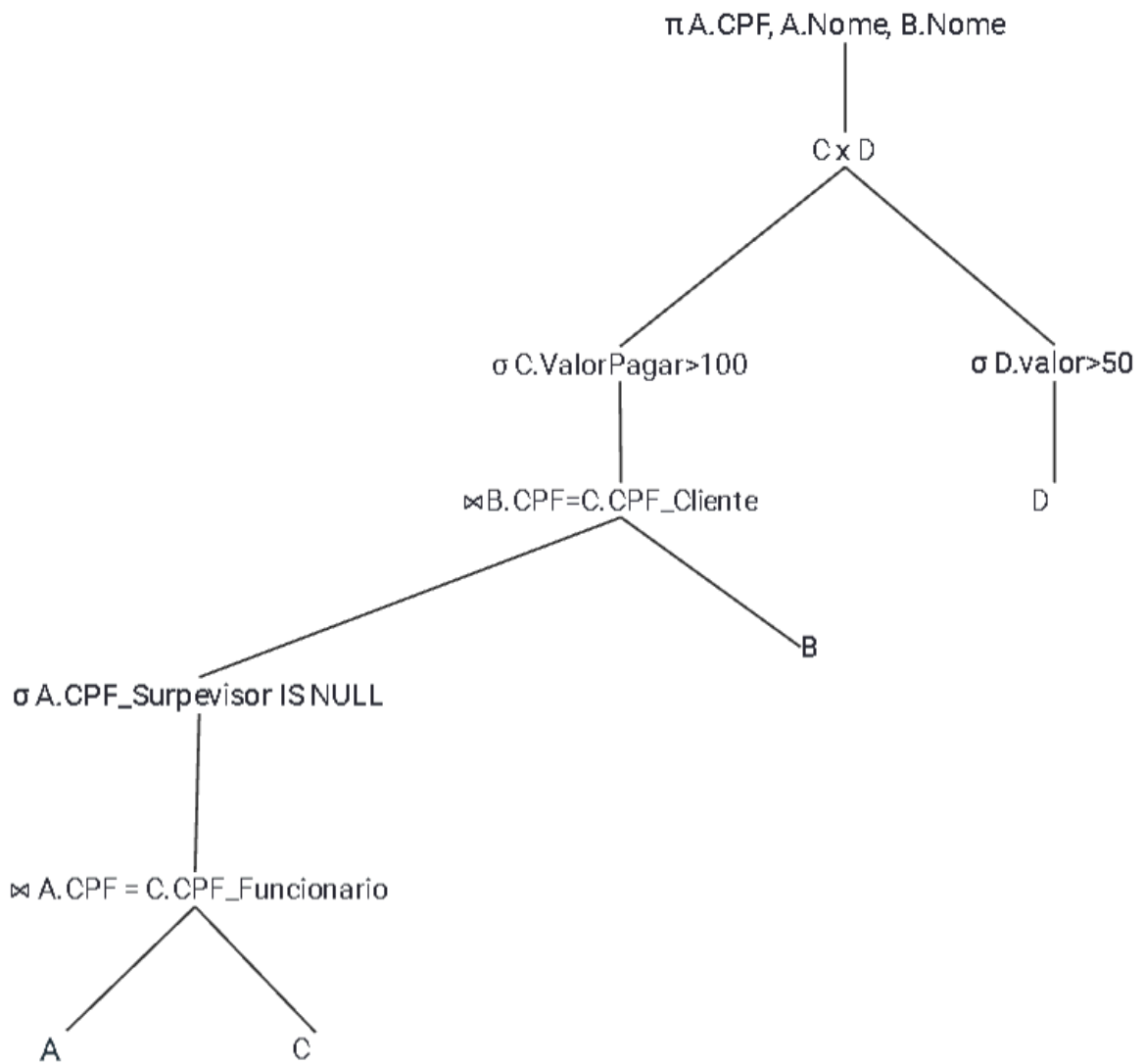
AND A.CPF_Supervisor IS NULL

AND A.CPF=C.CPF_Funcionario

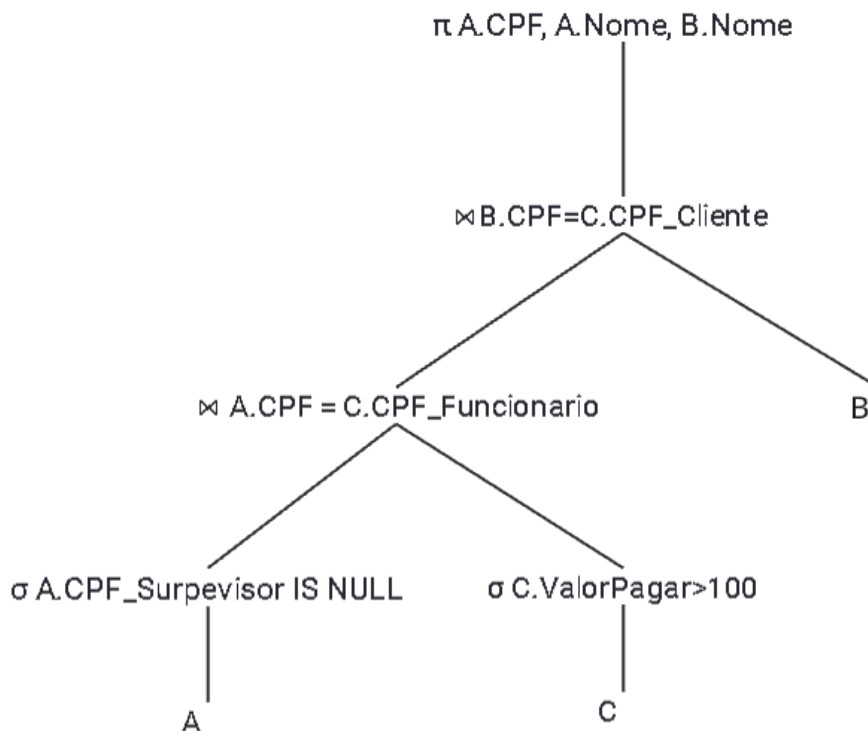
(1) Apresentar a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem natural (da esquerda para a direita):



(2) a árvore de consulta inicial (não otimizada) com o *parsing* da consulta em ordem reversa (da direita para a esquerda):



(3) a árvore de consulta otimizada:



(4) a consulta reescrita de acordo com a árvore de consulta otimizada com o *parsing* da consulta em ordem natural (da esquerda para a direita):

```

SELECT A.CPF, A.Nome, B.Nome
FROM Funcionarios AS A
JOIN Aluguel AS C
  ON A.CPF = C.CPF_Funcionario
JOIN Clientes AS B
  ON B.CPF = C.CPF_Cliente
WHERE A.CPF_Supervisor IS NULL
AND C.ValorPagar > 100;

```

(5) a consulta reescrita de acordo com a árvore de consulta otimizada com o *parsing* da consulta em ordem reversa (da direita para a esquerda):

```

SELECT A.CPF, A.Nome, B.Nome
FROM (
  (
    Funcionarios AS A
    JOIN Aluguel AS C
      ON A.CPF = C.CPF_Funcionario
    JOIN Clientes AS B
      ON B.CPF = C.CPF_Cliente
  )
)

```

)
WHERE A.CPF_Supervisor IS NULL
AND C.ValorPagar > 100;

(6) o plano de execução da consulta otimizada para cada uma das consultas SQL apresentadas abaixo:

1. Pesquisa Linear → arquivo não indexado Funcionarios (A)
2. Pesquisa Linear → arquivo não indexado Aluguel (C)
3. Junção de loop único → varre arquivo não indexado Funcionarios (A) e pesquisa arquivo de índice de Aluguel (C)
4. Pesquisa Linear → arquivo não indexado Clientes (B)
5. Junção de loop único → varre resultado intermediário ($A \bowtie C$) e pesquisa arquivo de índice de Clientes (B)
6. Projeção final (π) - Colunas: A.CPF, A.Nome, B.Nome